KANNADA SANGHA PUNE'S

KAVERI COLLEGE OF ARTS, SCIENCE & COMMERCE

A PROJECT REPORT ON

# DEALSCOUT

E-Commerce Product Comparison and Best-Deal Recommendation System

**TYBSC(CS) (SEM – V) • Academic Year 2026–27**

Name 1: _____ Seat No.: _____

Name 2: _____ Seat No.: _____

In partial fulfilment of B. Sc. (Computer Science)

# KAVERI COLLEGE OF ARTS, SCIENCE & COMMERCE

PERMANENTLY AFFILIATED TO SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

# CERTIFICATE

This is to certify that Mr. / Ms. _____, T.Y. BSc (Computer Science) Semester V, has successfully completed his/her Project Work titled “DealScout” in Computer Science Specialization as a part of CS-331-FP Project as prescribed by Savitribai Phule Pune University for academic year 2026–27.

Seat No.: _____ Date: _____

| **Project Guide** | **Course Coordinator** | **Principal** |
|---|---|---|
| Name: _____ | Name: _____ | Name: _____ |
| Signature: _____ | Signature: _____ | Signature: _____ |

## Examined by

| **Internal Examiner** | **External Examiner** |
|---|---|
| Name: _____ | Name: _____ |
| Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ |

# DECLARATION

I/We hereby declare that the work presented in this report entitled “DealScout” is an original study completed for T.Y. B.Sc. (Computer Science), Semester V.

- The implementation and documentation are prepared for academic evaluation.
- All external references are acknowledged in the bibliography.
- Student names and seat numbers are intentionally left editable.

Placeholders should be completed and signed before final submission. The declaration confirms ownership of the submitted work while recognizing official documentation and software references.

# ACKNOWLEDGEMENT

The completion of DealScout was made possible through guidance, review, testing, and access to academic resources.

- Gratitude to the Principal and Department of Computer Science.
- Thanks to the Project Guide and Course Coordinator.
- Acknowledgement of internal and external examiners.
- Appreciation for classmates, family, and open-source communities.

Replace this text with personal acknowledgements if required by the assigned teacher. Keep the tone formal, concise, and connected to the actual support received.

# INDEX — PART I


| | | |
|---|---|---|
| 1 | Preliminary Investigation — 1.1 Problem Identification; 1.2 Problem Statement; 1.3 Purpose, Objectives and Goals; 1.4 Feasibility Study; 1.5 Scope and Limitations | **8–15** |
| 2 | Requirement Specification — 2.1 System Requirements; 2.2 Technical Requirement; 2.3 Functional Requirements; 2.4 Data Requirements | **16–25** |
| 3 | Database Design — 3.1 End Users; 3.2 Entities, Attributes and ER Diagram; 3.3 Structures and Relationships | **26–32** |


Instructor signature: _____

Instructor signature: _____

Instructor signature: _____

# INDEX — PART II


| | | |
|---|---|---|
| 4 | System Design — Class, Object, Component, Deployment, Use Case, Activity and Sequence Diagrams | **33–40** |

Instructor signature: _____

| | | |
|---|---|---|
| 5 | Input and Output Screen Layout — Login, Registration, Add Record, Dashboard, Search Result and Insights | **41–46** |

Instructor signature: _____

| | | |
|---|---|---|
| 6 | Testing, Conclusion, Bibliography and Appendix | **47–50** |

Instructor signature: _____

# ABSTRACT

DealScout is a web application for comparing offers for the same product across multiple sellers. It combines search, normalization, recommendation, login, and search-history analysis in one responsive interface.

- Accepts product names and category filters.
- Normalizes price, seller, rating, discount, and delivery fields.
- Highlights the lowest listed price.
- Stores searches for later insights.
- Supports a sample catalog and an optional live shopping provider.

The solution reduces repeated manual searching. Its design separates the browser interface, REST API, data storage, and external provider adapter so that each part can be explained and maintained independently.

# 1.1 PROBLEM IDENTIFICATION

E-commerce platforms present similar products with different prices, discounts, ratings, seller names, and delivery estimates. Users must inspect each source separately.

- Manual comparison consumes time.
- Different field formats create confusion.
- A low visible price may exclude context such as seller or delivery.
- Search history is normally scattered or unavailable.
- Users need a transparent explanation of the recommended deal.

DealScout addresses these difficulties by displaying every normalized offer in one table and marking the minimum listed price. It does not claim that a preview price is live when no provider is connected.

# 1.2 PROBLEM STATEMENT / DEFINITION

To design and develop a responsive web system that searches product offers, converts them into a common structure, compares important values, identifies the lowest listed price, and stores search activity.

- Input: product query, category, and source mode.
- Process: validation, retrieval, normalization, minimum-price selection, and logging.
- Output: best-deal card, comparison table, notices, and insights.
- Constraint: live results require a valid provider key.

The problem definition is intentionally precise. It focuses on comparable offer data rather than promising purchase completion, payment handling, or guaranteed final checkout totals.

# 1.3 PURPOSE, OBJECTIVES AND GOALS

The purpose is to help users compare product offers with less effort and more context.

- Provide a clean and accessible search form.
- Support mobile, laptop, and general-product categories.
- Display seller, price, rating, delivery, and discount.
- Calculate the lowest price using understandable logic.
- Protect insights behind a login session.
- Keep the application portable through containerized setup.

Success is measured by whether a user can move from a query to a readable comparison and understand why one offer is highlighted. The interface must remain usable on desktop and mobile screens.

# 1.4 TECHNICAL FEASIBILITY

The required technologies are available, documented, and suitable for a student-scale application.

- React and TypeScript render the responsive interface.
- FastAPI provides validated JSON endpoints.
- MongoDB stores search logs.
- Docker Compose packages frontend, backend, and database.
- An optional SerpApi adapter supplies live shopping results.

The sample catalog guarantees that core functionality works without paid credentials. The application uses a typed HTTP boundary, clear data models, and server-side storage, making the architecture technically achievable.

# 1.4 ECONOMIC FEASIBILITY

Development can be completed with open-source tools and standard computer hardware.

- No license fee for React, Python, FastAPI, or MongoDB Community.
- Docker Desktop is free for eligible educational use under its terms.
- The sample catalog has no API cost.
- Hosting and live-provider costs are optional and depend on usage.

Economic feasibility is high because the MVP can run locally. The cost of internet access and a live provider is separated from the essential comparison workflow.

# 1.4 OPERATIONAL AND SCHEDULE FEASIBILITY

The application uses familiar search and table patterns, while the work can be divided into achievable stages.

- Operational: clear labels, visible loading states, and readable errors.
- Operational: sign-in is needed only for private insights.
- Schedule: requirements and data model first.
- Schedule: backend, frontend, testing, and documentation follow.
- Schedule: teacher feedback is applied before submission.

The modular sequence reduces project risk. Each stage produces a demonstrable output and supports independent correction without redesigning the entire system.

# 1.5 PROJECT SCOPE

DealScout covers product discovery and comparison for mobiles, laptops, and general items.

- Search a predefined sample catalog.
- Optionally retrieve live shopping results.
- Compare normalized offers.
- Highlight the minimum price.
- Store and summarize search activity.
- Provide login, responsive design, downloadable resources, and one-click setup.

The scope ends before checkout. DealScout is an information and decision-support application, not a retailer, payment processor, inventory authority, or delivery service.

# 1.5 LIMITATIONS

Every comparison system depends on the quality, freshness, and completeness of its source data.

- Sample prices are not current retailer prices.
- Live mode requires a configured provider key.
- Taxes, exact variant, location, and shipping may affect final checkout cost.
- The demo login is not production identity management.
- The current catalog contains a limited number of products.
- Retailer links may be unavailable in sample mode.

These limitations are shown honestly in the interface. Future work can add verified retailer partnerships, currency normalization, variant matching, and production authentication.

# 2.1 HARDWARE REQUIREMENTS

DealScout is lightweight enough for an ordinary student computer.

- Processor: dual-core 64-bit processor or better.
- Memory: minimum 4 GB RAM; 8 GB recommended for Docker.
- Storage: at least 3 GB free for images, dependencies, and database volume.
- Display: 1280×720 or higher recommended.
- Network: optional for sample mode, required for dependency download and live provider.

Docker resource settings may be adjusted for older machines. Once images are built, the sample catalog and database can run on a local network without external API access.

# 2.1 SOFTWARE REQUIREMENTS

The portable setup uses Docker to avoid separate runtime installation.

- Operating system: Windows 10/11, macOS, or modern Linux.
- Docker Desktop or Docker Engine with Compose.
- Modern browser: Chrome, Edge, Firefox, or Safari.
- Development stack: Node.js 24, Python 3.12, Nginx, and MongoDB 7 inside containers.
- Code editor is optional for normal use.

The setup scripts call Docker Compose, build the services, start them in the background, and expose DealScout at localhost port 3000.

# 2.2 FRONTEND TECHNICAL REQUIREMENTS

The frontend must be responsive, type-safe, accessible, and independent of hard-coded backend addresses.

- React components for pages and reusable interface blocks.
- TypeScript interfaces mirror API response models.
- TanStack Query manages server data and mutations.
- Tailwind CSS and shadcn components provide consistent styling.
- Relative /api paths work locally and behind the Nginx proxy.

The interface renders its shell even when server data is unavailable. Interactive controls include clear data-testid values to support browser verification.

# 2.2 BACKEND TECHNICAL REQUIREMENTS

The backend must validate requests, use asynchronous database calls, and expose every route under /api.

- FastAPI APIRouter organizes authentication, products, and analytics.
- Pydantic models validate requests and responses.
- Motor provides asynchronous MongoDB access.
- HTTPX connects to the optional shopping provider.
- Environment variables store connection settings and provider secrets.

The server never exposes the provider key to browser code. Route modules share one database handle and return JSON structures that match frontend interfaces.

# 2.2 DATABASE AND DEPLOYMENT REQUIREMENTS

Search activity must survive service restarts and remain isolated from application source files.

- MongoDB volume stores search logs.
- Indexes support recent-search sorting.
- Docker Compose starts MongoDB before application use.
- Nginx serves the frontend and proxies /api requests.
- The setup supports an optional SERPAPI_KEY environment variable.

Although the college guideline describes relational tables, the implementation uses equivalent document collections. The report identifies entities, fields, keys, and relationships clearly.

# 2.3 FUNCTIONAL REQUIREMENTS — ACCESS

Access requirements define how visitors and signed-in users move through the application.

- FR-01: Visitor can open the comparison page.
- FR-02: Visitor can open the sign-in page.
- FR-03: Valid credentials create an HTTP-only session cookie.
- FR-04: Invalid credentials return a clear error.
- FR-05: Signed-in user can view shopping insights.
- FR-06: Guest cannot retrieve protected analytics data.

Authentication protects personal search summaries while leaving the core comparison experience publicly accessible.

# 2.3 FUNCTIONAL REQUIREMENTS — COMPARISON

The central functional requirements describe the complete search-to-recommendation flow.

- FR-07: Accept a query between 2 and 80 characters.
- FR-08: Filter by All, Mobiles, Laptops, or General.
- FR-09: Select Sample prices or Live search.
- FR-10: Display normalized offers.
- FR-11: Identify the offer with minimum price.
- FR-12: Show a truthful notice when live data is unavailable.
- FR-13: Record every completed search.

The highlighted result is based on listed price only. The UI retains seller, delivery, rating, and discount so the user can make a broader judgment.

# 2.4 DATA REQUIREMENTS

Data requirements define the minimum information needed to compare offers and summarize searches.

- User: id, name, email, and role.
- ProductOffer: id, product name, category, source, seller, price, original price, rating, delivery, discount, URL, mode, flags, and retrieval time.
- SearchLog: id, query, category, mode, result count, best price, sources, and search time.
- AnalyticsSummary: totals, top query, source frequency, and recent searches.

String UUID values are used as application identifiers. Prices use numeric values and timestamps are generated on the server.

# 2.4 PERFORMANCE REQUIREMENTS

The application should feel responsive for a small educational workload.

- Local sample searches should normally return within one second.
- Live provider requests use a finite timeout.
- Search responses are limited to a small result set.
- Recent analytics reads are capped at 100 records.
- The UI displays loading text during network work.
- Responsive layouts avoid unnecessary horizontal scrolling except wide tables.

Performance depends on hardware and provider availability. The design avoids unbounded database reads and does not block startup for index creation.

# 2.4 SECURITY REQUIREMENTS

Security requirements protect credentials, sessions, provider keys, and server data.

- Provider secrets stay in backend environment variables.
- Sessions use HTTP-only cookies.
- Protected analytics validates the active session.
- Request bodies and query lengths are validated.
- Frontend never stores authentication tokens.
- Production deployments should use HTTPS, persistent sessions, password hashing, and restricted CORS.

The bundled login is explicitly a preview account. It demonstrates access control but must be replaced before handling real user identities.

# CHAPTER 3 — DATABASE DESIGN AND END USERS

Database design identifies the people who use DealScout and the entities required by their actions.

- Visitor searches and compares offers.
- Signed-in shopper reviews insights.
- Future administrator manages catalog and provider settings.
- Evaluator or maintainer reviews behavior and records.

User roles are deliberately simple. Permissions are applied at the analytics endpoint, while comparison remains available without registration.

# 3.2 ENTITIES AND ATTRIBUTES

The data model contains User, ProductOffer, SearchLog, and AnalyticsSummary entities.

- User primary key: id.
- ProductOffer primary key: id.
- SearchLog primary key: id.
- SearchLog sources represent stores included in a comparison.
- AnalyticsSummary is calculated from SearchLog records rather than stored separately.

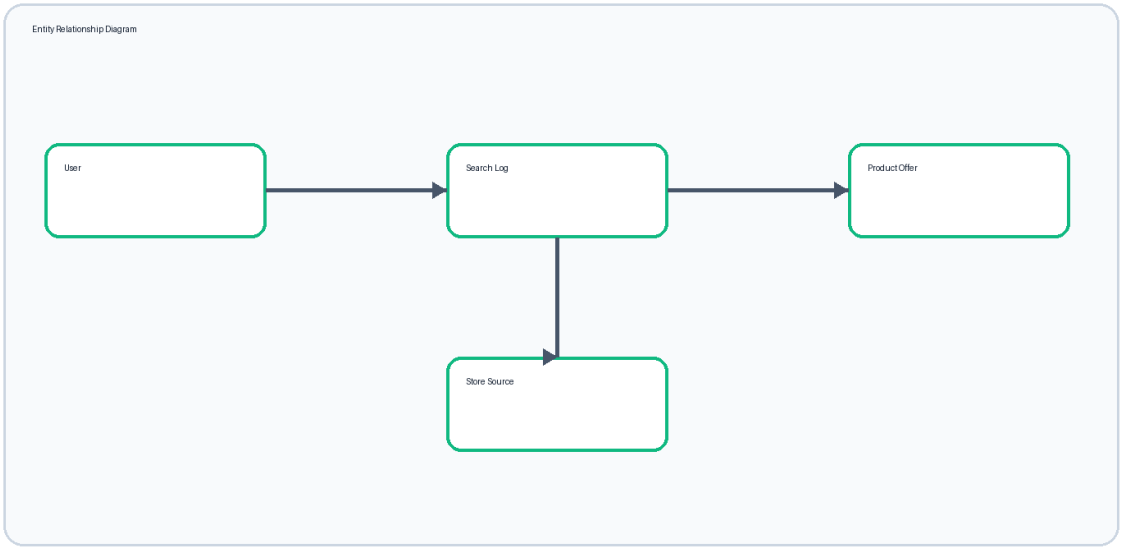
Entity attributes correspond directly to validated backend models and handwritten frontend interfaces, reducing ambiguity across the HTTP boundary.

# 3.2 ER DIAGRAM

The ER diagram shows the logical relationship between a shopper, searches, offers, and store sources.

- A user can review many search logs.
- A search can return many product offers.
- An offer belongs to one source at retrieval time.
- Analytics aggregates many search logs.

The current preview does not persist a direct user foreign key on search logs; adding owner_id is recommended for a true multi-user production version.



# 3.3 USER TABLE / COLLECTION

The User model represents an authenticated identity used to open protected insights.

- id — string, primary application identifier.
- name — string, display name.
- email — string, login identifier.
- role — string, access role.

The preview user is configured in server memory. A production implementation should use a users collection with unique email index, password hash, created time, and account status.

# 3.3 PRODUCT OFFER STRUCTURE

ProductOffer is the normalized structure used for every seller result.

- price and original_price are numeric.
- currency defaults to INR.
- rating, delivery, discount, and URL may be absent.
- mode identifies sample or live retrieval.
- is_live and is_mocked prevent misleading labels.
- retrieved_at communicates freshness.

Normalization allows a single comparison table and minimum-price function to work across different source schemas.

# 3.3 SEARCH LOG STRUCTURE

SearchLog records completed comparisons for later analysis.

- id — unique string.
- query — submitted product text.
- category — selected filter.
- mode — demo or live.
- result_count — number of normalized offers.
- best_price — minimum price or null.
- searched_at — UTC server timestamp.
- sources — source names returned by the search.

The searched_at descending index supports the recent-comparison view. In production, owner_id should be indexed with searched_at.

# 3.3 RELATIONSHIPS AND DATA FLOW

The browser submits a query, the backend produces ProductOffer objects, and one SearchLog is inserted before the response returns.

- Logical User-to-SearchLog relationship: one-to-many.
- SearchLog-to-ProductOffer relationship: one-to-many during a response.
- Source-to-ProductOffer relationship: one-to-many.
- Analytics-to-SearchLog relationship: calculated aggregation.

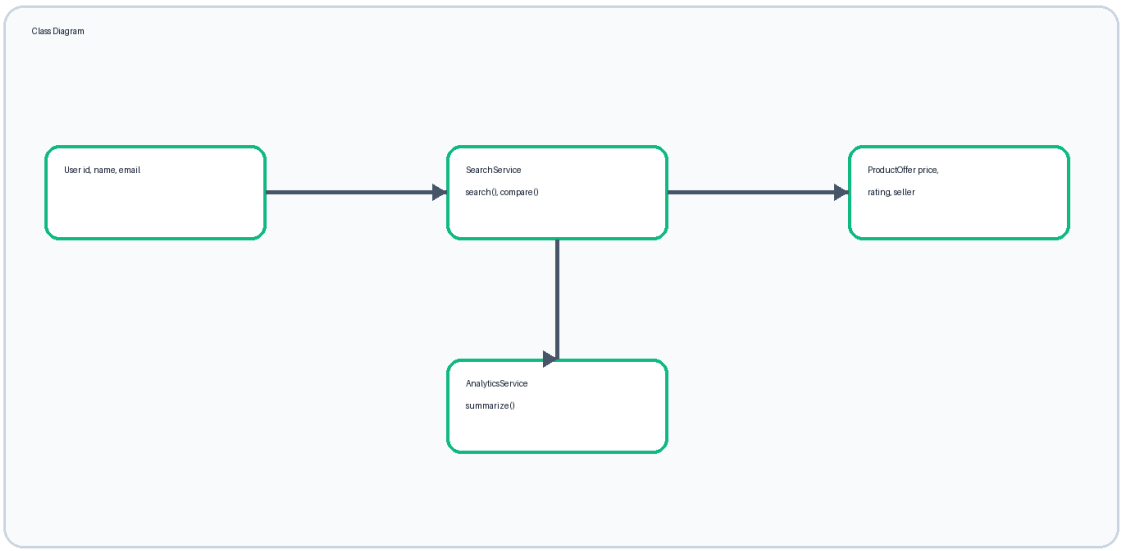
The application stores summary search data rather than full third-party payloads. This keeps the database compact and limits unnecessary retention.

# 4.1 CLASS DIAGRAM

The class diagram connects data models to the services that use them.

- User contains identity and role data.
- ProductOffer contains comparable fields.
- SearchService retrieves, normalizes, and compares.
- AnalyticsService summarizes logs.
- Authentication functions create and validate sessions.

The diagram separates responsibilities so that UI rendering, business logic, and storage remain understandable.

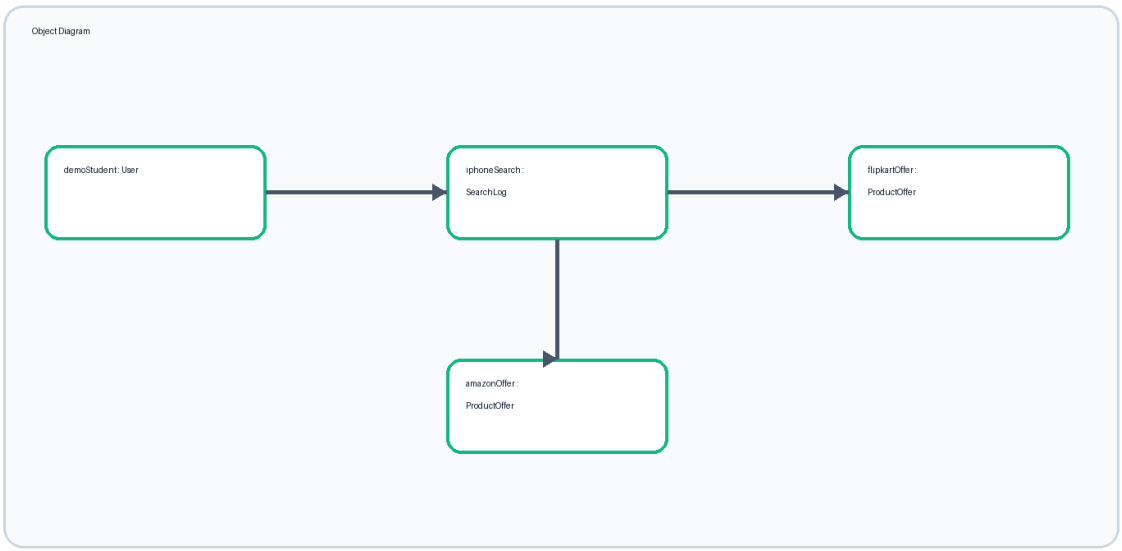


# 4.2 OBJECT DIAGRAM

The object diagram illustrates a single iPhone comparison at a moment in time.

- demoStudent is an instance of User.
- iphoneSearch is an instance of SearchLog.
- flipkartOffer and amazonOffer are ProductOffer instances.
- The search object references returned source values.

Object diagrams help explain how class definitions become actual runtime values.

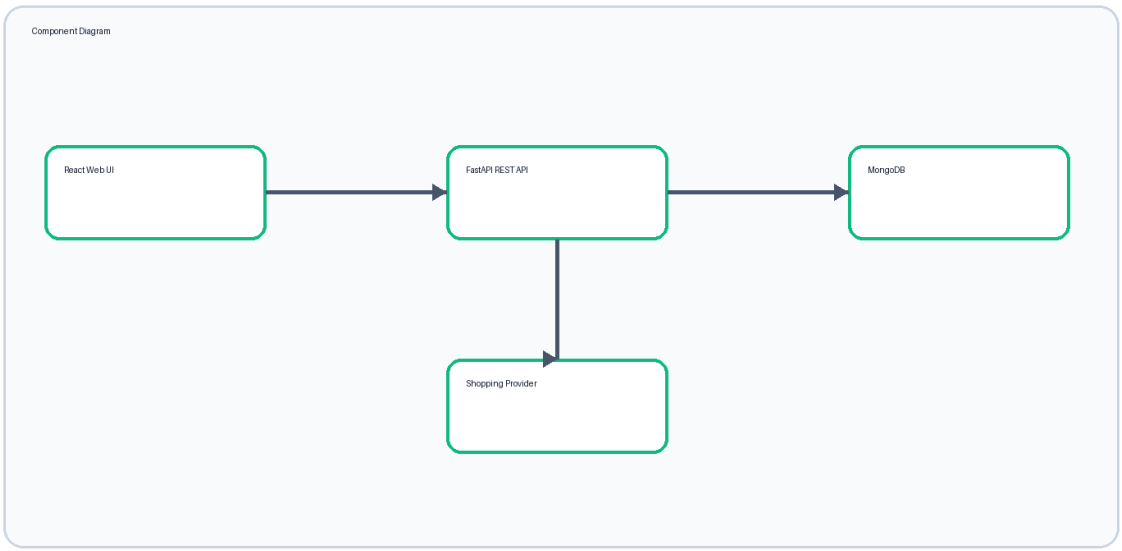


# 4.3 COMPONENT DIAGRAM

The component diagram displays the major deployable or logical software modules.

- React web UI handles presentation and interaction.
- FastAPI REST API validates and coordinates work.
- MongoDB persists search logs.
- Shopping provider optionally supplies live offers.
- Nginx serves static files and proxies /api.

All browser-server communication uses JSON over relative /api routes.

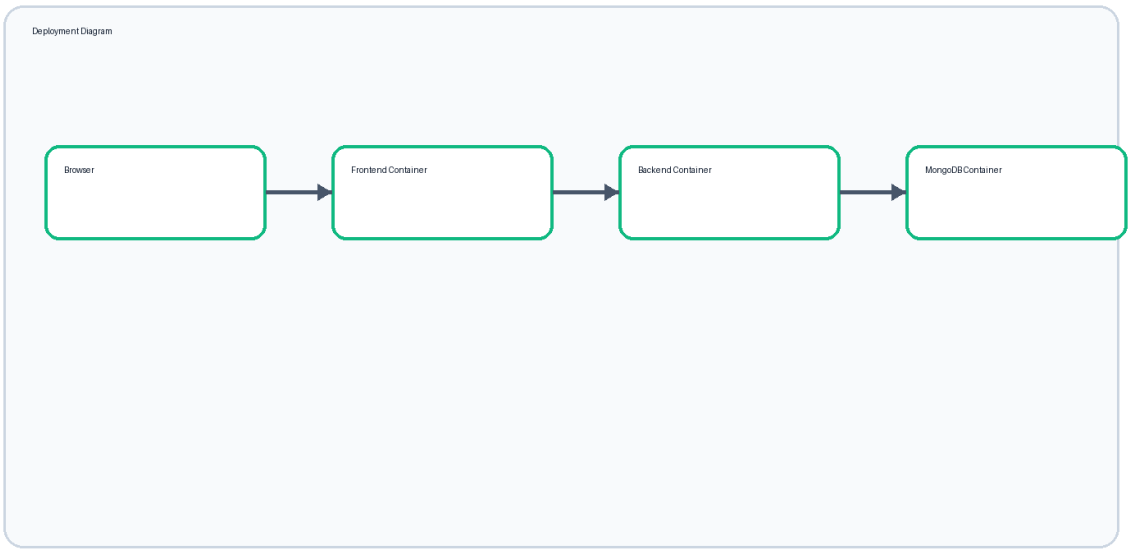


# 4.4 DEPLOYMENT DIAGRAM

Docker Compose runs DealScout as three cooperating services on one machine.

- Frontend container exposes port 3000.
- Backend container listens internally on 8001.
- MongoDB container stores data in a named volume.
- The browser reaches only the frontend origin.
- The backend reaches MongoDB and the optional provider.

The same layout can run on Windows, macOS, Linux, or a compatible server.

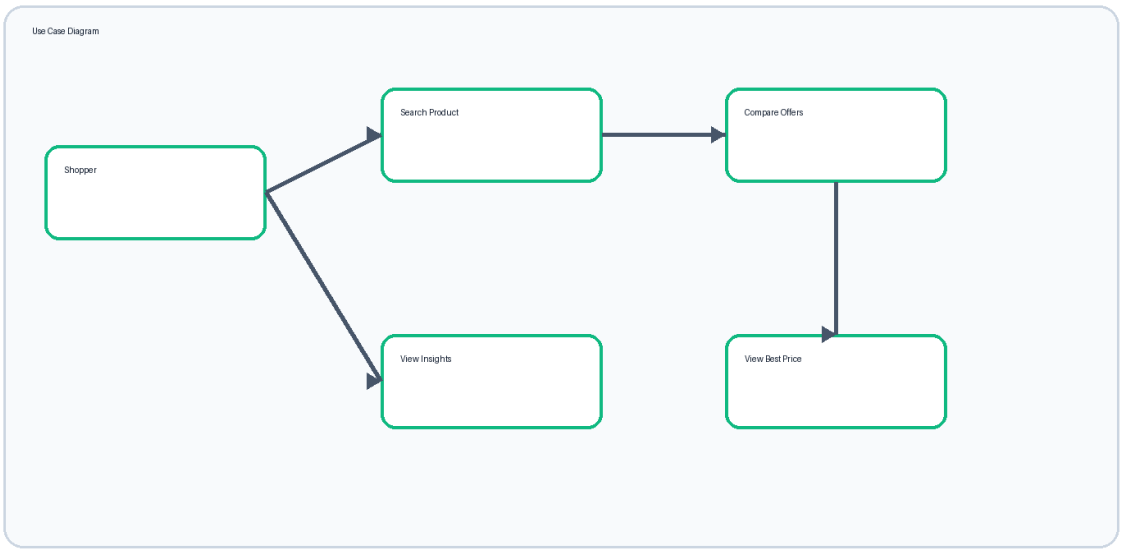


# 4.5 USE CASE DIAGRAM

Use cases describe behavior from the shopper's point of view.

- Search product.
- Choose category and data source.
- Compare normalized offers.
- View best price.
- Sign in.
- View comparison insights.
- Download support resources.

Advanced administration and purchasing are outside the current scope.

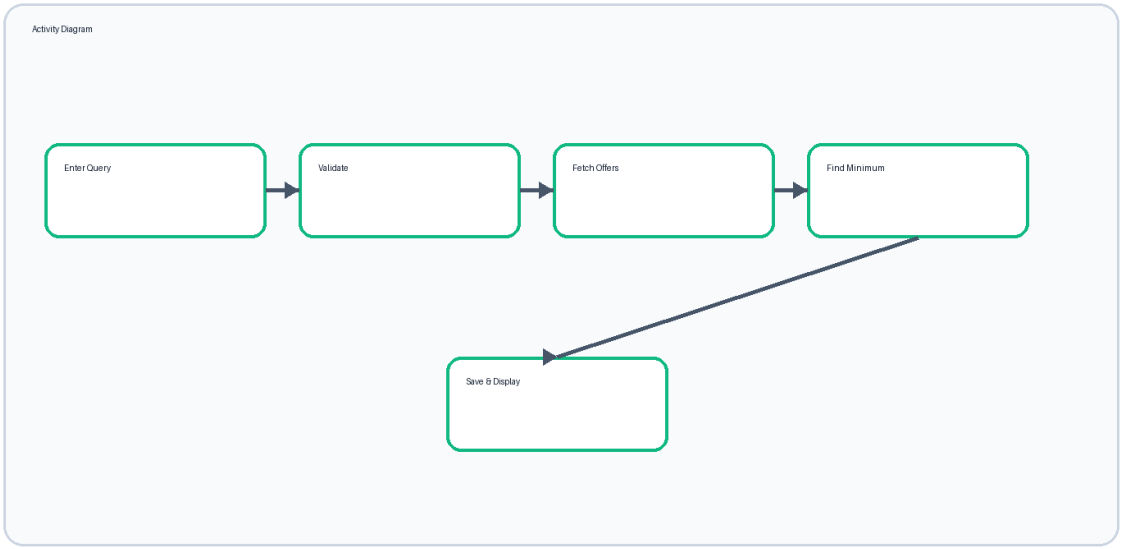


# 4.6 ACTIVITY DIAGRAM

The main activity begins with user input and ends with a displayed and recorded comparison.

- Enter and validate query.
- Read sample catalog or request provider.
- Normalize accepted results.
- Find minimum price.
- Insert search log.
- Display notice, best-price card, and offer table.

If no offers match, the UI displays a useful empty state instead of a blank page.

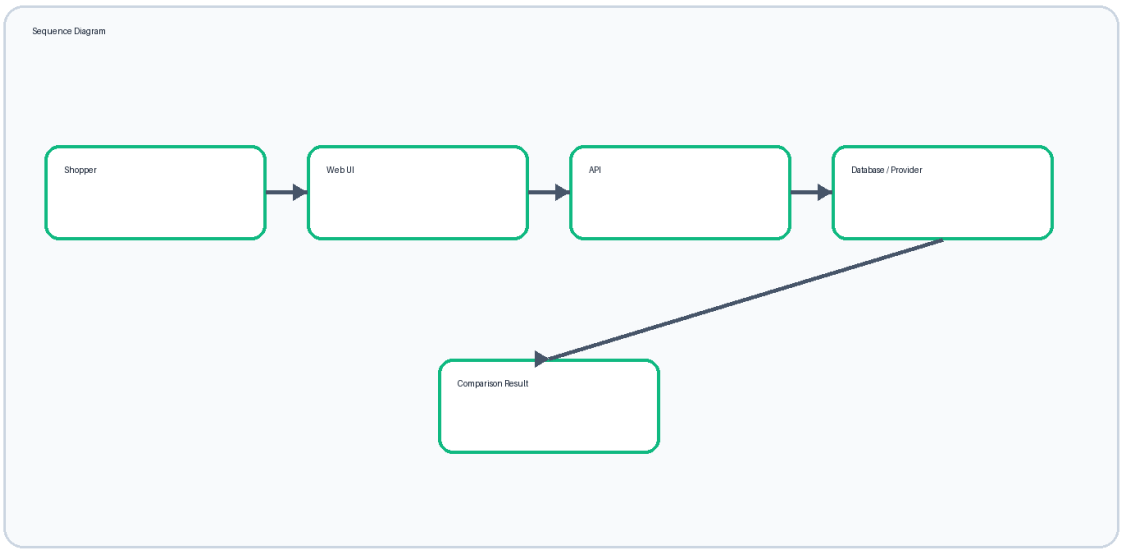


# 4.7 SEQUENCE DIAGRAM

The sequence diagram emphasizes message order between shopper, UI, API, database, and provider.

- UI sends GET /api/products/search.
- API optionally calls the provider.
- API constructs ProductOffer models.
- API saves a SearchLog.
- API returns SearchResponse JSON.
- UI renders and later invalidates analytics cache.

Login follows a similar request sequence but creates an HTTP-only cookie.



# SYSTEM ARCHITECTURE AND ALGORITHM

DealScout uses a layered client-server architecture and a simple minimum-price recommendation algorithm.

- Presentation layer: responsive React pages.
- API layer: validation and routing.
- Business layer: matching, normalization, and comparison.
- Data layer: MongoDB search logs.
- Integration layer: optional shopping API adapter.

Algorithm: collect matching offers; discard entries without a usable price; compare numeric price values; select min(results, key=price); return its id as best_deal_id; render that row and card with emerald emphasis.

# 5.1.1 LOGIN SCREEN

The login screen grants access to private shopping insights.

- Email input validates browser email format.
- Password input masks entered characters.
- Sign-in button submits credentials.
- Invalid credentials display a concise error.
- Preview account details support evaluation.

The visual layout separates product value messaging from the focused sign-in card.

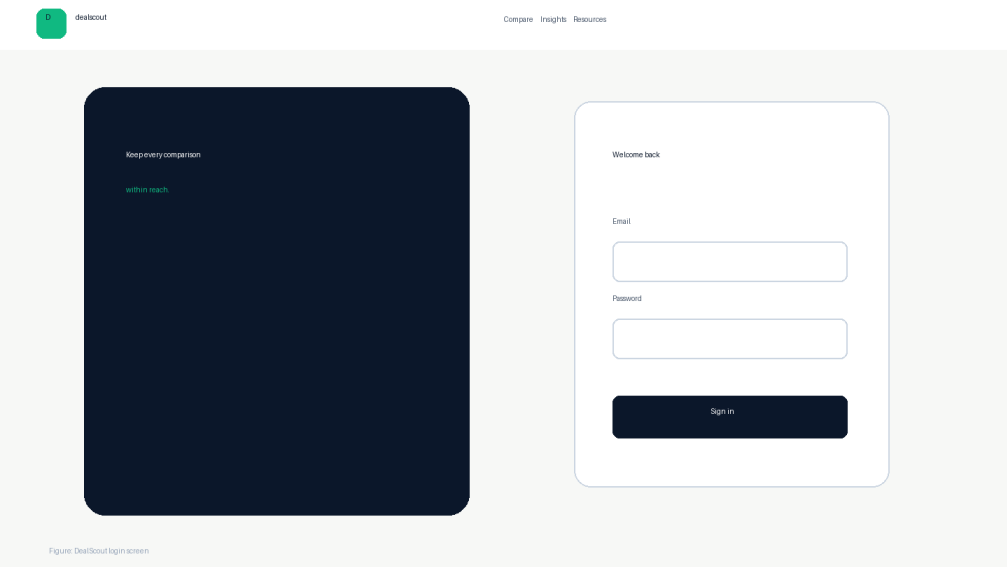



Figure: DealScout login screen

# 5.1.2 REGISTRATION SCREEN

Registration is not implemented in the current preview because only one evaluation account is required.

- Proposed fields: full name, email, password, confirm password.
- Proposed validation: unique email and password policy.
- Proposed output: account creation confirmation and active session.
- Production passwords must be hashed, never stored directly.

This documented future screen satisfies the planned multi-user extension without falsely claiming that registration is available today.

# 5.1.3 ADD RECORD SCREEN

Catalog editing is currently controlled in backend data rather than exposed to public users.

- Proposed administrator fields: product name, category, store, seller, price, original price, rating, delivery, and URL.
- Required validation: positive price, recognized category, safe URL, and unique source offer.
- Actions: save, clear, cancel, and preview.

An authenticated administrator role and audit log should be introduced before this screen is enabled.

# 5.2.1 DASHBOARD

The dashboard gives immediate access to brand navigation, mode selection, category filter, product query, examples, and feature explanation.

- Hero communicates the service benefit.
- Search form is visually dominant.
- Sample and Live controls are clearly separated.
- Popular queries accelerate first use.
- Responsive cards explain the three-step flow.

The page avoids academic language so it appears as a professional consumer product.

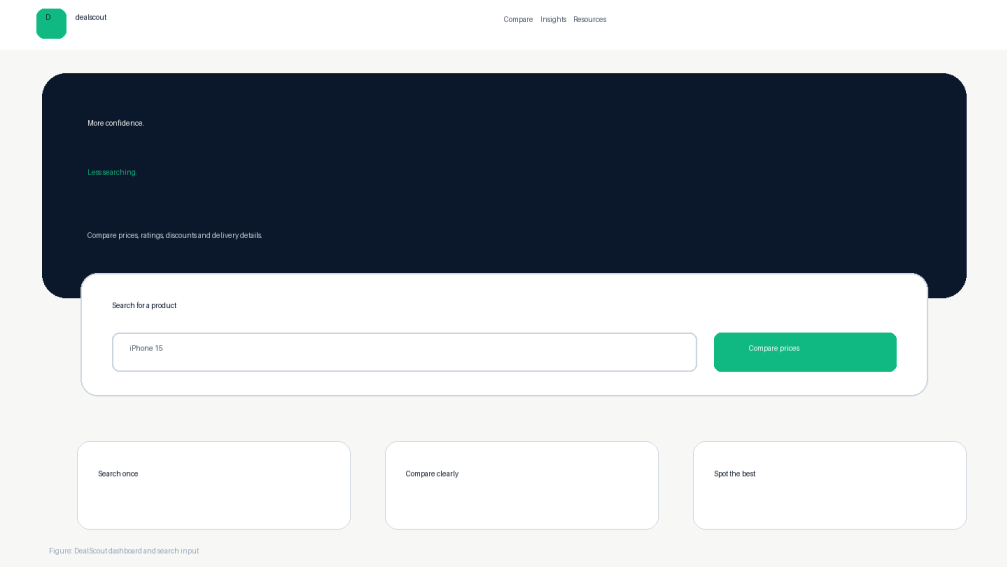



Figure: DealScout dashboard and search input

# 5.2.2 SEARCH RESULT SCREEN

Search results place the best price before the detailed comparison table.

- Query, result count, and source label remain visible.
- Preview-mode notice prevents misleading live claims.
- Best-price card identifies product, store, seller, delivery, price, and discount.
- Table supports horizontal scrolling on narrow screens.
- No-results state suggests shorter queries.

Emerald is reserved for savings and the best-price recommendation.

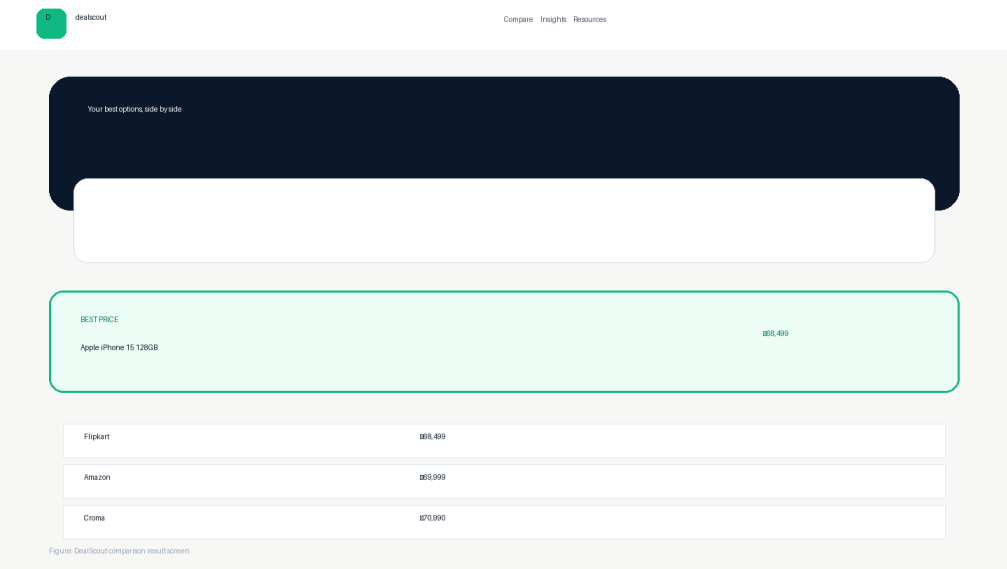


Figure: DealScout comparison result screen

# 5.2.3 REPORT / INSIGHTS SCREEN

The insights screen summarizes stored comparison activity for a signed-in user.

- Comparison count.
- Unique search count.
- Most searched query.
- Recent comparison table.
- Store frequency list.
- Sign-in prompt for guests.

The screen uses compact cards and tables rather than decorative charts so values remain easy to verify.

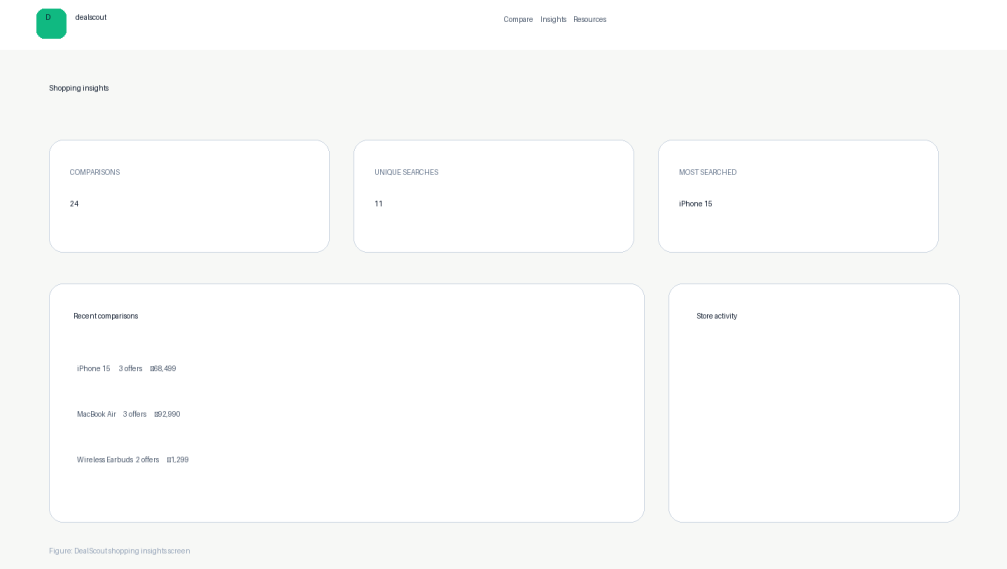



Figure: DealScout shopping insights screen

# TESTING AND VALIDATION

Testing confirms the main flow and important failure states across API, type system, and browser.

- Public API root returns DealScout status.
- Valid search returns results and best_deal_id.
- One-character query returns HTTP 422.
- Valid login returns user and sets cookie.
- Protected analytics requires login.
- Frontend strict TypeScript check passes.
- Browser flow searches, signs in, and opens insights without console errors.

A change is considered complete only when the exact failed check passes after correction.

# CONCLUSION AND FUTURE SCOPE

DealScout demonstrates a complete comparison workflow with a professional responsive interface, validated API, data persistence, access control, downloadable resources, and portable setup.

- Connect verified retailer or shopping-provider credentials.
- Add product-variant matching and currency normalization.
- Include shipping and tax in total-cost comparison.
- Replace preview login with persistent users and hashed passwords.
- Add price alerts, favorites, and historical trends.
- Create an administrator catalog editor.

The current solution meets its defined objective while clearly separating implemented behavior from future enhancements.

# CHAPTER 6 — BIBLIOGRAPHY / REFERENCES

The following sources support implementation and documentation.


- FastAPI Documentation — https://fastapi.tiangolo.com/
- React Documentation — https://react.dev/
- MongoDB Documentation — https://www.mongodb.com/docs/
- Docker Documentation — https://docs.docker.com/
- MDN Web Docs — https://developer.mozilla.org/
- SerpApi Google Shopping API — https://serpapi.com/google-shopping-api
- SPPU and Kaveri College documentation instructions supplied for academic year 2026–27.


All product names and retailer names are used for demonstration. Live data use must follow the chosen provider's terms, rate limits, attribution, and caching rules.

# APPENDIX — SUBMISSION AND VIVA CHECKLIST

This final page helps prepare the report and system before review.

- Fill student names, seat numbers, guide, coordinator, principal, examiner, date, and signatures.
- Ask the assigned teacher to review formatting and diagrams.
- Update index page numbers after final edits.
- Open the public or local app and test search, login, and insights.
- Explain sample data versus optional live data honestly.
- Carry the Word report, PDF report, viva guide, source code, and setup guide.

Core viva explanation: DealScout receives a query, normalizes offers to one model, finds the minimum numeric price, stores a search log, and displays both the recommendation and supporting comparison rows.